

Fruit Quality Classification

Federico Cesare Cattò

Gennaio 2026

Contents

1	Dataset Overview	2
1.1	Label Extraction	2
1.2	Quality Class Distribution	2
1.3	Fruit Type Distribution	3
1.4	Sample Images by Quality	3
2	Data Preprocessing	5
2.1	Dataset Preparation and DataLoaders	6
3	Model Training & Evaluation	7
3.1	Baseline Model	7
3.2	Advanced Model – EfficientNet-B0	8
3.3	Test Set Evaluation – Advanced Model	10
3.4	Performance Differences Between Validation and Test Sets	11
4	Grad-CAM Analysis	13
5	Conclusions	14

Chapter 1

Dataset Overview

The dataset employed for this task was obtained by aggregating multiple datasets collected from the Dataverse Harvard repository. Each individual dataset contains images of a single fruit type and a single quality level, resulting in a highly controlled annotation structure.

The combined dataset consists of a total of 1,484 images and includes both common fruit types, such as apples and strawberries, as well as less frequent varieties, including mangoes, lemons, and pomegranates. Images are organized in a structured directory hierarchy, and each filename encodes both the fruit type and its quality label, for example `Apple_good_(1).jpg`.

1.1 Label Extraction

To extract the labels, a preprocessing function was implemented that parses the fruit type and quality from the filenames. The procedure involved removing the file extension and any trailing index, and then splitting the remaining string by underscores (`_`) to separate the fruit type from the quality label. The resulting dataset is represented as a DataFrame with the following structure:

filename	fruit	quality
Apple_good_(1).jpg	Apple	good
Strawberry_bad_(3).jpg	Strawberry	bad
Papaya_imperfect_(2).jpg	Papaya	imperfect

Table 1.1: Example of parsed labels for the fruit dataset.

1.2 Quality Class Distribution

Analysis of the quality labels indicates that the dataset is imbalanced. The majority of images are labeled as *good* and *imperfect*, while a smaller proportion are labeled as *bad*. The detailed distribution is reported in Table 1.2, and a corresponding bar plot visualizes these differences.

The predominance of good-quality images suggests that care must be taken during model training to ensure that the classifier does not become biased toward the majority class, particularly when distinguishing the minority class representing poor-quality fruits.

Quality	Count	Percentage
good	764	51.48%
imperfect	646	43.53%
bad	74	4.99%

Table 1.2: Distribution of fruit quality classes.

1.3 Fruit Type Distribution

The dataset comprises 16 distinct fruit types. Apples are the most represented with 897 images, whereas oranges and some other fruit types have fewer than 20 images each. Figure 1.1 summarizes the counts per fruit type, and the accompanying bar chart provides a visual overview.

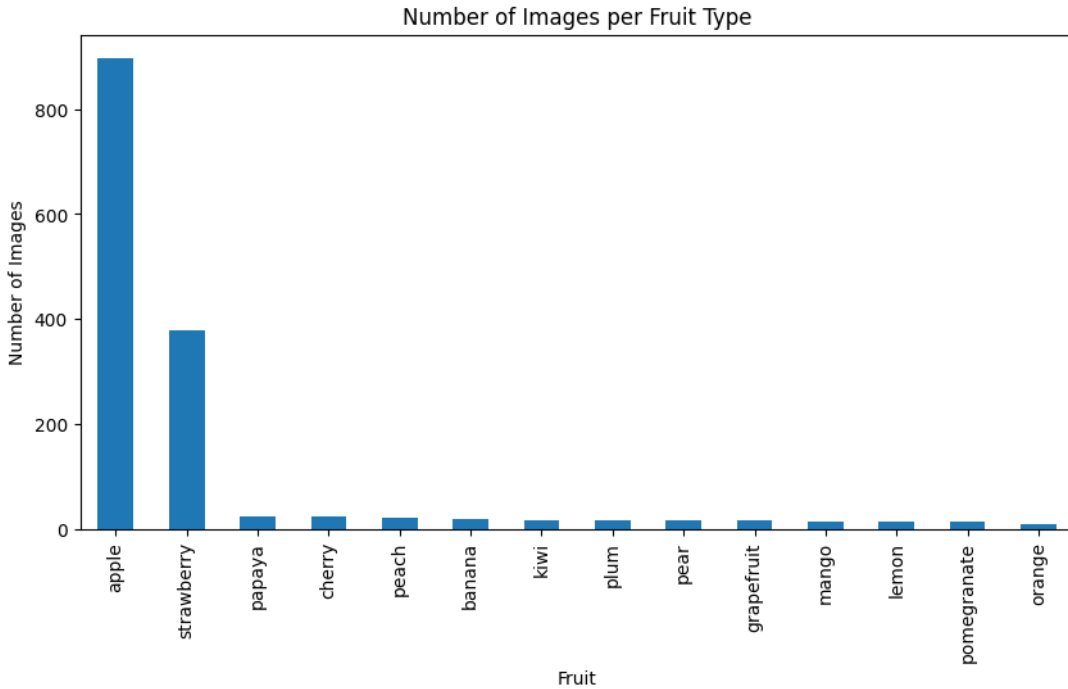


Figure 1.1: Number of images per fruit type.

The uneven representation of fruit types implies that, during model training, techniques such as data augmentation or stratified sampling may be required to prevent poor classification performance on underrepresented fruits.

1.4 Sample Images by Quality

A selection of sample images from each quality category illustrates the visual characteristics used for labeling. Images labeled as *good* display intact fruits with vibrant colors and no visible defects. *Imperfect* fruits exhibit minor blemishes or discoloration, while *bad* fruits show clear damage or decay. The examples in Figures 1.2, 1.4, 1.3 demonstrate that the quality labels align with intuitive human perception, which is critical for supervised learning applications.

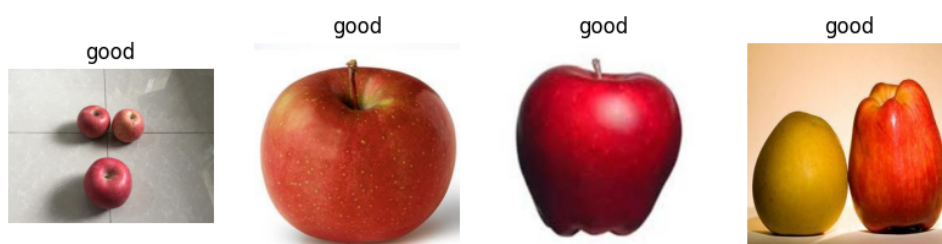


Figure 1.2: Example images for each fruit quality category.



Figure 1.3: Example images for each fruit quality category.



Figure 1.4: Example images for each fruit quality category.

Chapter 2

Data Preprocessing

Prior to model training, all images underwent a preprocessing pipeline to ensure quality, consistency, and compatibility with convolutional neural networks pretrained on ImageNet. The original images were first inspected for corruption. Each image was opened and verified to ensure that no file was damaged or unreadable. All 1,484 images in the dataset were found to be valid, confirming the integrity of the dataset.

Next, the images were standardized to the RGB color space and resized to a uniform dimension of 224×224 pixels, a common requirement for standard convolutional architectures. To improve numerical stability during training and to leverage pretrained feature representations, the pixel values were normalized using the mean and standard deviation from the ImageNet dataset:

$$\text{mean} = [0.485, 0.456, 0.406], \quad \text{std} = [0.229, 0.224, 0.225]$$

For the training set, a comprehensive set of data augmentation techniques was applied to artificially increase the diversity of the dataset and to improve the robustness of the model. Augmentation simulated real-world variations in fruit orientation, lighting conditions, and partial occlusions. The following transformations were employed:

- **Random horizontal flip** to account for symmetrical appearances of fruits.
- **Random rotation** within $\pm 20^\circ$ to handle variable orientation.
- **Color jittering**, including variations in brightness, contrast, saturation, and hue, to mimic differences in illumination and ripeness.
- **Random resized crop** to emulate variability in fruit position and scale within images.

The combination of these augmentations ensures that the model learns robust features rather than overfitting to specific appearances or positions in the training set.

For validation and test images, no augmentation was applied. Instead, images were simply converted to RGB, resized to 224×224 pixels, and normalized using the ImageNet statistics. This maintains consistency between the training and evaluation pipelines while avoiding artificial alterations of evaluation images.

Overall, this preprocessing and augmentation pipeline enhances the effective size and diversity of the dataset, preparing it for effective training of transfer learning models while mitigating risks of overfitting.



Figure 2.1: Example of augmented images generated from a single sample image. Augmentation simulates variations in orientation, lighting, and position, improving model robustness.

2.1 Dataset Preparation and DataLoaders

The dataset was organized to facilitate efficient training and evaluation of deep learning models. Initially, all image filenames were parsed to extract the fruit type and quality label. The filenames encode both pieces of information, for example `Apple_good_(20).jpg`, from which the fruit type “Apple” and quality “good” were extracted. Quality labels were then mapped to numerical values as follows: `good` = 0, `imperfect` = 1, and `bad` = 2.

To prepare the dataset for training and evaluation, a stratified split was performed to preserve the class distribution across all subsets. Specifically, the dataset was divided into a training set (70% of images), a validation set (15%), and a test set (15%). The stratified sampling ensured that the proportion of each quality class remained approximately constant across all splits:

- **Training set:** good 51.4%, imperfect 43.5%, bad 5.0%
- **Validation set:** good 51.6%, imperfect 43.5%, bad 4.9%
- **Test set:** good 51.6%, imperfect 43.5%, bad 4.9%

A custom PyTorch `Dataset` class was implemented to handle image loading, preprocessing, and label assignment. Each image is loaded from disk, converted to RGB format, and transformed according to the preprocessing pipeline defined previously. For the training set, data augmentation was applied, whereas validation and test images underwent only resizing and normalization. The corresponding quality label is returned alongside the image tensor.

To facilitate model training, PyTorch `DataLoader` objects were created for each subset, enabling mini-batch processing. A batch size of 32 was used. The training `DataLoader` applies shuffling to improve generalization, while validation and test `DataLoaders` are kept fixed to ensure reproducible evaluation. This modular setup cleanly separates data handling from model training, allowing scalable and maintainable experimentation with different architectures and training strategies.

Chapter 3

Model Training & Evaluation

3.1 Baseline Model

A baseline model was implemented to serve as a reference for subsequent experiments involving more advanced architectures and fine-tuning strategies. We employed a pre-trained *ResNet18* network, using its convolutional layers as a fixed feature extractor. All convolutional parameters were frozen during training, and only the final fully connected layer was replaced with a new linear layer outputting three classes corresponding to fruit quality: **good**, **imperfect**, and **bad**.

The model was trained using the Adam optimizer with a learning rate of 10^{-3} and cross-entropy loss. Training proceeded for five epochs, with performance evaluated on the validation set after each epoch. The pipeline leveraged the previously defined DataLoaders and preprocessing/augmentation strategies.

The per-class metrics on the validation set are reported in Table 3.1, with aggregated metrics in Table 3.3 and overall accuracy in Table 3.2.

Class	Precision	Recall	F1-score	Support
good	0.89	0.95	0.92	115
imperfect	0.92	0.91	0.91	97
bad	0.75	0.27	0.40	11

Table 3.1: Per-class metrics for the baseline ResNet18 model. The *bad* class is highlighted due to low recall.

Metric	Value
Accuracy	0.90

Table 3.2: Overall accuracy of the baseline ResNet18 model.

Metric	Precision	Recall	F1-score	Support
Macro avg	0.85	0.71	0.74	223
Weighted avg	0.89	0.90	0.89	223

Table 3.3: Aggregated metrics of the baseline ResNet18 model.

The normalized confusion matrix (Figure 3.1) confirms strong performance for **good** and **imperfect** classes, with most errors occurring between these adjacent categories. The

bad class is often misclassified due to the limited number of samples and subtle visual differences.

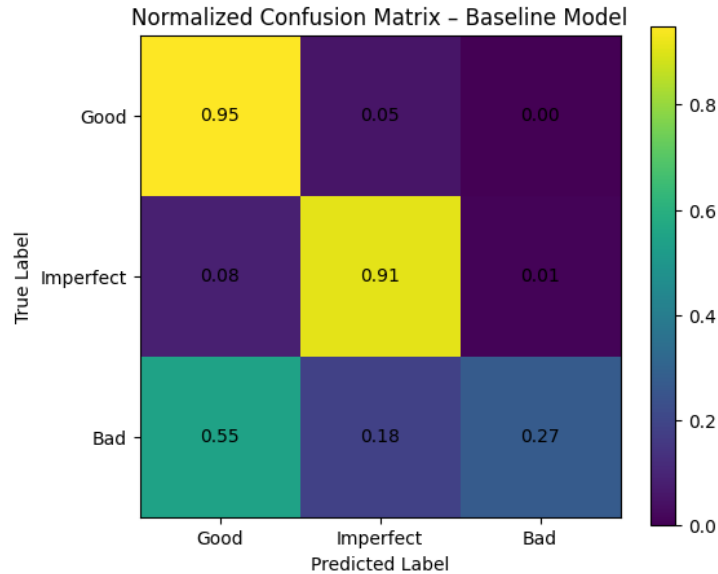


Figure 3.1: Normalized confusion matrix for the baseline ResNet18 model.

Overall, the baseline model effectively distinguishes **good** and **imperfect** fruits, reflecting the relative ease of identifying high-quality fruits versus minor defects. Detection of severely defective (**bad**) fruits remains limited, with many misclassified as **good** or **imperfect**. This outcome is expected due to the small number of **bad** images and the use of a frozen feature extractor, motivating the exploration of fine-tuning and class-aware training strategies.

3.2 Advanced Model – EfficientNet-B0

To enhance detection of underrepresented and challenging classes, we implemented an advanced model based on *EfficientNet-B0* using transfer learning. The network was initialized with pre-trained weights, and the final classification layer was modified to output three classes: **good**, **imperfect**, and **bad**.

Initially, the convolutional backbone was frozen and only the classifier head was trained. Training employed a weighted cross-entropy loss, where weights were inversely proportional to class frequency to penalize errors on underrepresented classes, particularly **bad** fruits. Optimization used AdamW with a learning rate of 10^{-3} and weight decay of 10^{-4} , while a *ReduceLROnPlateau* scheduler dynamically adjusted the learning rate if validation loss plateaued.

After initial training, the last convolutional blocks of EfficientNet-B0 were unfrozen for fine-tuning with a reduced learning rate of 10^{-4} to prevent catastrophic forgetting. This two-stage approach—head training followed by selective fine-tuning—enables the model to leverage pre-trained features while adapting to the specific dataset.

Training lasted a total of 15 epochs (10 initial + 5 fine-tuning), with validation performance monitored throughout. Per-class, aggregated, and overall metrics are reported in Tables 3.4, 3.6, and 3.5.

Class	Precision	Recall	F1-score	Support
good	0.93	0.97	0.95	115
imperfect	0.98	0.90	0.94	97
bad	0.79	1.00	0.88	11

Table 3.4: Per-class metrics for the advanced EfficientNet-B0 model. The *bad* class is highlighted to show improved detection.

Metric	Value
Accuracy	0.94

Table 3.5: Overall accuracy of the advanced EfficientNet-B0 model.

Metric	Precision	Recall	F1-score	Support
Macro avg	0.90	0.96	0.92	223
Weighted avg	0.95	0.94	0.94	223

Table 3.6: Aggregated metrics of the advanced EfficientNet-B0 model.

The normalized confusion matrix (Figure 3.2) shows that the advanced model achieves perfect recall for the **bad** class, demonstrating the effectiveness of class-weighted loss and fine-tuning. Misclassifications between **good** and **imperfect** remain minimal.

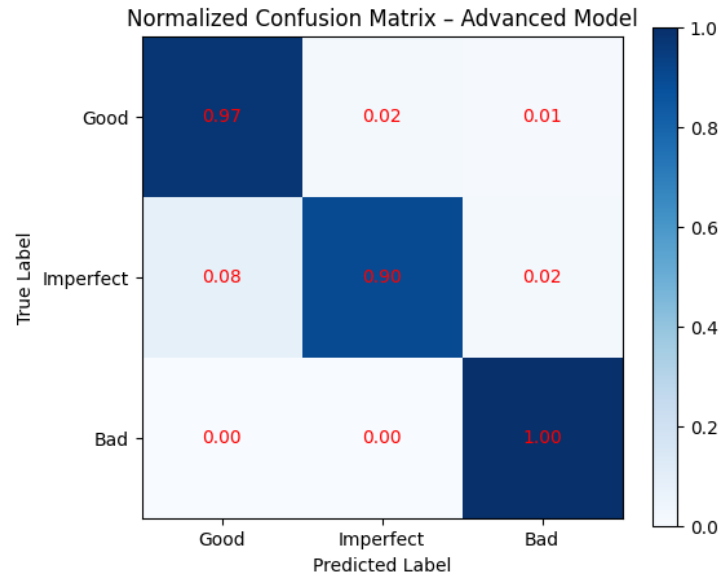


Figure 3.2: Normalized confusion matrix for the EfficientNet-B0 model. Perfect recall for **bad** fruits demonstrates the benefit of class-weighted loss and fine-tuning.

These results indicate that the advanced model outperforms the baseline, particularly in identifying severely defective fruits. Leveraging transfer learning, class-aware loss, and selective fine-tuning allows the network to maintain high overall accuracy while effectively detecting minority classes.

3.3 Test Set Evaluation – Advanced Model

The advanced EfficientNet-B0 model was evaluated on the held-out test set to assess generalization performance. Standard classification metrics—including precision, recall, F1-score, and overall accuracy—are summarized in Tables 3.7, 3.9, and 3.8.

Class	Precision	Recall	F1-score	Support
good	0.95	0.96	0.95	115
imperfect	0.95	0.90	0.92	97
bad	0.47	0.64	0.54	11

Table 3.7: Per-class metrics of the advanced EfficientNet-B0 model on the test set. The *bad* class is highlighted due to low F1-score.

Metric	Value
Accuracy	0.91

Table 3.8: Overall accuracy of the advanced EfficientNet-B0 model on the test set.

Metric	Precision	Recall	F1-score	Support
Macro avg	0.79	0.83	0.80	223
Weighted avg	0.92	0.91	0.92	223

Table 3.9: Aggregated metrics of the advanced EfficientNet-B0 model on the test set.

The normalized confusion matrix (Figure 3.3) indicates strong performance for **good** and **imperfect** fruits, while **bad** fruits remain more challenging due to their limited number and subtle defects.

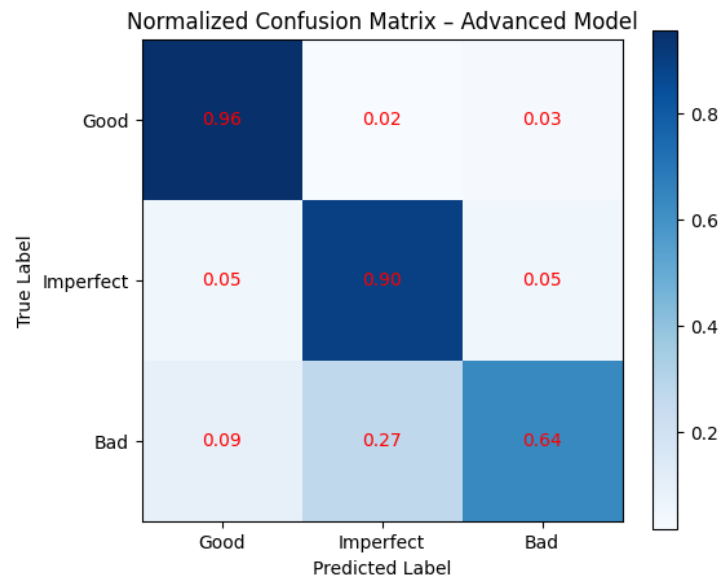


Figure 3.3: Normalized confusion matrix for the advanced model on the test set. Most errors occur between **imperfect** and **bad** classes.

Analysis of false positives and false negatives per class reveals typical error patterns:

- **good**: 6 false positives, 5 false negatives
- **imperfect**: 5 false positives, 10 false negatives
- **bad**: 8 false positives, 4 false negatives

Inspection of misclassified images shows that most errors arise under challenging conditions such as unusual lighting, occlusions, or subtle defects that are difficult even for human observers to detect. An example is shown in Figure 3.4.



Figure 3.4: Example of a misclassified **bad** fruit. Challenging conditions a dark color background.

In conclusion, the advanced transfer learning model substantially improves recognition of severely defective fruits while maintaining high accuracy on majority classes, providing a robust foundation for fruit quality classification.

3.4 Performance Differences Between Validation and Test Sets

It is observed that the advanced EfficientNet-B0 model achieves higher performance metrics on the validation set compared to the test set, particularly for the **bad** fruit class. This behavior is expected and can be explained by several factors.

First, during training, the model’s performance on the validation set is monitored repeatedly to guide hyperparameter selection and early stopping. As a result, the model can become indirectly tuned to the validation set, leading to slightly optimistic performance estimates. In contrast, the test set remains completely unseen until the final evaluation, providing a more reliable measure of generalization.

Second, the **bad** class is underrepresented, with only 11 samples in both validation and test sets. In small classes, a few misclassifications have a large impact on precision, recall, and F1-score. On the validation set, the model correctly identifies all **bad** samples, yielding perfect recall (1.00). On the test set, however, several **bad** fruits are misclassified due to subtle visual defects, variations in lighting, or occlusions, resulting in a lower recall (0.64) and precision (0.47).

This discrepancy highlights the challenges of detecting rare and visually subtle classes and underscores the importance of using weighted loss functions, data augmentation, or oversampling to improve generalization. Overall, while validation metrics provide a useful guide during training, the test set offers a more realistic assessment of the model’s ability to handle unseen and difficult examples.

Chapter 4

Grad-CAM Analysis

To interpret the decisions made by the advanced EfficientNet-B0 model, Grad-CAM (Gradient-weighted Class Activation Mapping) was applied. Grad-CAM highlights the regions of an input image that contributed most to the model's prediction, providing a visual explanation of its reasoning process.

In the selected example, the model classified the fruit as **Good**. The resulting heatmap, overlaid on the original image, shows that areas with warmer colors (red and yellow) correspond to regions that the network considered most informative for its decision. Specifically, the activation is concentrated along the edges and surface of the fruit, indicating that the model relies on visual cues such as shape and texture to assess quality.

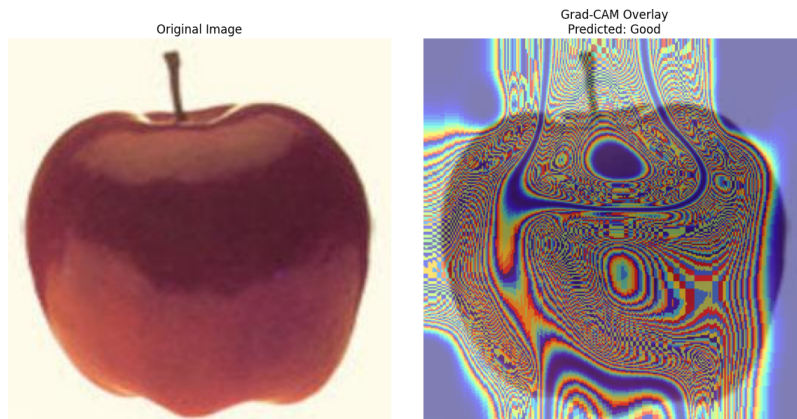


Figure 4.1: Grad-CAM visualization for a fruit classified as **Good**. Left: original image. Right: Grad-CAM overlay highlighting regions that influenced the model's prediction. The model focuses on relevant fruit areas, ignoring background elements such as leaves or hands.

The overlay presents a characteristic "interference" pattern or contour-like map over the fruit, typical of activation visualizations that track gradient flow through convolutional filters. This analysis confirms that the network is learning meaningful features related to fruit quality rather than relying on irrelevant background information.

Chapter 5

Conclusions

In this chapter, we presented a complete workflow for fruit quality classification using image-based deep learning. The dataset consisted of 1,484 images spanning 16 fruit types and three quality categories, exhibiting natural class imbalance and uneven representation across fruit types. A preprocessing and augmentation pipeline was implemented to standardize image dimensions, normalize pixel values, and increase dataset diversity, improving model robustness against variations in orientation, lighting, and occlusions.

We initially trained a baseline ResNet18 model using frozen convolutional layers, which achieved high accuracy for the majority classes (**good** and **imperfect**) but struggled to correctly identify severely defective fruits (**bad**) due to their limited number and subtle visual features. This motivated the adoption of an advanced transfer learning approach based on EfficientNet-B0, employing a weighted cross-entropy loss and a two-stage fine-tuning strategy. The advanced model substantially improved detection of the minority **bad** class while maintaining high overall accuracy, demonstrating the effectiveness of class-aware training and selective fine-tuning.

Evaluation on the held-out test set highlighted typical challenges in minority class detection, with a lower F1-score for **bad** fruits due to limited samples and subtle defects. Grad-CAM visualizations confirmed that the model focuses on relevant fruit regions for quality assessment, indicating that learned features are semantically meaningful.

Overall, these results demonstrate that transfer learning with class-aware strategies can provide robust and interpretable models for fruit quality classification, even in small, imbalanced datasets. Future work could explore:

- Collecting additional images for underrepresented fruits and low-quality categories to improve generalization.
- Applying advanced data augmentation or generative techniques to synthetically enrich minority classes.
- Investigating multi-task models that jointly predict fruit type and quality to exploit cross-class correlations.
- Deploying lightweight models suitable for real-time classification in industrial or agricultural settings.

In conclusion, the project provides a practical demonstration of combining data pre-processing, transfer learning, and interpretability techniques to build effective image classification pipelines for quality assessment tasks.